

AFRILANG Stdlib

std/c/mlnearest

Français. Bibliothèque AFRILANG 'std/c/mlnearest' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : nearSum, nearMean, nearMin, nearMax, nearVar, nearStd, nearNorm.

```
'import "std/c/mlnearest"' · 'use mlnearest'
```

```
- 'nearSum(v list number) → number' - 'nearMean(v list number) → number' - 'nearMin(v list number) → number' - 'nearMax(v list number) → number' - 'nearVar(v list number) → number' - 'nearStd(v list number) → number' - 'nearNorm(v list number) → list number'
```

English.

AFRILANG library 'std/c/mlnearest' (tier complex, category complex). Exports 7 function(s): nearSum, nearMean, nearMin, nearMax, nearVar, nearStd, nearNorm.

```
'import "std/c/mlnearest"' · 'use mlnearest'
```

```
- 'nearSum(v list number) → number' - 'nearMean(v list number) → number' - 'nearMin(v list number) → number' - 'nearMax(v list number) → number' - 'nearVar(v list number) → number' - 'nearStd(v list number) → number' - 'nearNorm(v list number) → list number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlnearest' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : nearSum, nearMean, nearMin, nearMax, nearVar, nearStd, nearNorm.

```
'import "std/c/mlnearest"' · 'use mlnearest'
```

```
- `nearSum(v list number) → number`
- `nearMean(v list number) → number`
- `nearMin(v list number) → number`
- `nearMax(v list number) → number`
- `nearVar(v list number) → number`
- `nearStd(v list number) → number`
- `nearNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlnearest"
use mlnearest
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- nearSum(v list number) → number•
- nearMean(v list number) → number•
- nearMin(v list number) → number•
- nearMax(v list number) → number•
- nearVar(v list number) → number•
- nearStd(v list number) → number•
- nearNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/mlnearest"
use mlnearest
```

```
create result = nearSum(/* args */)
say result
```

```
create result = nearMean(/* args */)
say result
```

```
create result = nearMin(/* args */)
say result
```

```
create result = nearMax(/* args */)
say result
```

```
create result = nearVar(/* args */)
say result
```

```
create result = nearStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlnearest"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlnearest

```
function nearSum(v list number) returns number
end
```

```
function nearMean(v list number) returns number
end
```

```
function nearMin(v list number) returns number
end
```

```
function nearMax(v list number) returns number
end
```

```
function nearVar(v list number) returns number
end
```

```
function nearStd(v list number) returns number
end
```

```
function nearNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlnearest' (tier complex, category complex). Exports 7 function(s): nearSum, nearMean, nearMin, nearMax, nearVar, nearStd, nearNorm.

'import "std/c/mlnearest"' · 'use mlnearest'

```
- `nearSum(v list number) → number`
- `nearMean(v list number) → number`
- `nearMin(v list number) → number`
- `nearMax(v list number) → number`
- `nearVar(v list number) → number`
- `nearStd(v list number) → number`
- `nearNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlnearest"
use mlnearest
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- nearSum(v list number) → number•
nearMean(v list number) → number•
nearMin(v list number) → number•
nearMax(v list number) → number•
nearVar(v list number) → number•
nearStd(v list number) → number•
nearNorm(v list number) → list

4. Usage examples

```
import "std/c/mlnearest"
use mlnearest
```

```
create result = nearSum(/* args */)
say result
```

```
create result = nearMean(/* args */)
say result
```

```
create result = nearMin(/* args */)
say result
```

```
create result = nearMax(/* args */)
say result
```

```
create result = nearVar(/* args */)
say result
```

```
create result = nearStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlnearest"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlnearest

```
function nearSum(v list number) returns number
end
```

```
function nearMean(v list number) returns number
end
```

```
function nearMin(v list number) returns number
end
```

```
function nearMax(v list number) returns number
end
```

```
function nearVar(v list number) returns number
end
```

```
function nearStd(v list number) returns number
end
```

```
function nearNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlnearest"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/mlnearest/>

Source (extrait)

```
module mlnearest

function nearSum(v list number) returns number
end

function nearMean(v list number) returns number
```

```
end

function nearMin(v list number) returns number
end

function nearMax(v list number) returns number
end

function nearVar(v list number) returns number
end

function nearStd(v list number) returns number
end

function nearNorm(v list number) returns list number
end
end
```